Design and Analysis of Algorithms (DAA)

Lab Manual — Lab 4

Smart Optimization Assistant using Dynamic Programming

Longest Common Subsequence · 0/1 Knapsack · Matrix Chain Multiplication

---

Complete Lab Report

Submitted by	Tamanna
Programme	BCA AI&DS
University	K.R. Mangalam University
Course	Design and Analysis of Algorithms (DAA)
Experiment	Lab 4 — Smart Optimization Assistant using Dynamic Programming (Mapped CO: CO3)
Repository name	daa-lab4-smart-optimization-assistant
Report date	7 November 2026

Contents

1. Introduction 4

2. Objectives 4

3. Problem Statement 4

4. Tools and Environment (Question 1 — Project Workspace Setup) 5

4.1 Workspace setup (Windows PowerShell) 5

4.2 Repository structure and GitHub 5

5. Dynamic Programming Concepts Used 5

6. Application Design 6

7. Module 1 — Sequence Analysis: Longest Common Subsequence (Question 2) . . . 6

7.1 Recurrence and algorithm 7

7.2 Implementation (from app.py) 7

7.3 Sample run: X = ABCBDAB, Y = BDCABA 7

8. Module 2 — Resource Allocation: 0/1 Knapsack (Question 3) 9

8.1 Recurrence and algorithm 9

8.2 Implementation (from app.py) 10

8.3 Sample run: weights 2,3,4,5 — values 3,4,5,6 — capacity 5 10

8.4 Why greedy fails for 0/1 Knapsack 12

9. Module 3 — Computation Optimization: Matrix Chain Multiplication (Question 4) . 13

9.1 Recurrence and algorithm 13

9.2 Implementation (from app.py) 13

9.3 Sample run: dimensions 10, 30, 5, 60 13

10. Complexity and Performance Analysis (Question 5) 16

10.1 Complexity comparison table 16

10.2 Measurements on the sample inputs 16

10.3 Measured scaling for growing input sizes 17

10.4 Measured growth against theory 19

10.5 Space usage 19

11. Comparison with the Naive Recursive Approach 20

12. Visualizations (Question 6) 21

13. Practical Applications 22

14. Key Observations 22

15. Limitations and Future Work 22

16. Conclusion 23

Appendix A — Complete Source Code 24

A.1 requirements.txt 24
A.2 app.py 25
Appendix B — Experiment Script 28

1. Introduction

Dynamic Programming (DP) solves a problem by breaking it into overlapping subproblems, solving each subproblem only once, and storing (tabulating) the result so it can be reused. This lab implements a **Smart Optimization Assistant**: an interactive Streamlit application that applies three classic DP algorithms to three kinds of real-world optimization problems.

Module	Algorithm	Kind of problem
Sequence Analysis	Longest Common Subsequence (LCS)	Finding what two sequences have in common (text, DNA, files)
Resource Allocation	0/1 Knapsack	Choosing the most valuable set of items within a fixed capacity or budget
Computation Optimization	Matrix Chain Multiplication (MCM)	Choosing the cheapest order in which to multiply a chain of matrices

For each module the application displays the optimal answer, the filled DP table (as a table and as a heatmap), and the execution time and peak memory of the run. A fourth tab summarizes the theoretical complexity. This report documents the design, the implementation, the experimental performance analysis, the comparison with naive recursion, practical applications and the main observations.

2. Objectives

- Implement LCS, 0/1 Knapsack and Matrix Chain Multiplication using bottom-up Dynamic Programming.
- Display the DP tables, the optimal solution and the intermediate computations through an interactive interface.
- Measure execution time, space usage and the number of subproblems solved, and prepare a complexity comparison table.
- Visualize DP tables and performance statistics for different problem sizes.
- Compare the DP approach with naive recursion and discuss practical applications and key observations.

3. Problem Statement

Develop a Smart Optimization Assistant using Dynamic Programming to solve real-world optimization problems involving sequence analysis, resource allocation and computation optimization. The system should implement Longest Common Subsequence (LCS), 0/1 Knapsack and Matrix Chain Multiplication algorithms to generate optimal solutions, visualize Dynamic Programming tables, and analyze algorithm performance. It should provide an interactive interface, display intermediate computations, and present results through visualizations, complexity analysis and performance reports.

4. Tools and Environment (Question 1 — Project Workspace Setup)

Tool / Library	Purpose in this project
Python 3.10 or newer	Implementation language for all algorithms
Streamlit	Interactive web interface (tabs, text inputs, buttons, metrics, tables)
NumPy	Converts DP tables to arrays for the heatmaps
Matplotlib	Heatmaps of the DP tables; graphs in this report
Pandas	Displays DP tables and the complexity summary as data frames
VS Code	Editor used for development
Git and GitHub	Version control; the repository holds code, screenshots, README and report

4.1 Workspace setup (Windows PowerShell)

```
mkdir daa_lab4
cd daa_lab4
# put app.py and requirements.txt in this folder
py -m venv venv
venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
pip install -r requirements.txt
python -m streamlit run app.py
```

If PowerShell blocks the activation script, run `Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass` once in the same terminal, then activate again. The contents of `requirements.txt` are the four libraries: streamlit, numpy, matplotlib and pandas.

4.2 Repository structure and GitHub

```
daa-lab4-smart-optimization-assistant/
├── app.py              Streamlit application (all three algorithms + UI)
├── requirements.txt    streamlit, numpy, matplotlib, pandas
├── README.md           setup and run instructions
└── screenshots/        screenshots of the running application

git init
git add .
git commit -m "Complete Lab 4 dynamic programming assistant"
git branch -M main
git remote add origin https://github.com/YOUR-USERNAME/daa-lab4-smart-optimization-assistant.git
git push -u origin main
```

5. Dynamic Programming Concepts Used

A problem can be solved with Dynamic Programming when it has two properties:

- **Optimal substructure** — the best solution to the whole problem is built from best solutions to smaller subproblems. For example, the best way to multiply $A_1 \ldots A_n$ contains best ways to multiply two shorter sub-chains.
- **Overlapping subproblems** — the same subproblems are needed again and again. A plain recursive solution recomputes them every time and becomes exponential; DP computes each one once and stores it.

This lab uses **bottom-up tabulation**: the table is filled from the smallest subproblems upward, so every value needed is already available when a cell is computed. The same five steps are used for every algorithm:

- define the **state** (what one table cell means);
- write the **recurrence** relating a cell to smaller cells;
- set the **base cases** (the first row and column);

- choose a **fill order** so dependencies are ready;
- **trace back** through the table to recover the actual solution, not just its value.

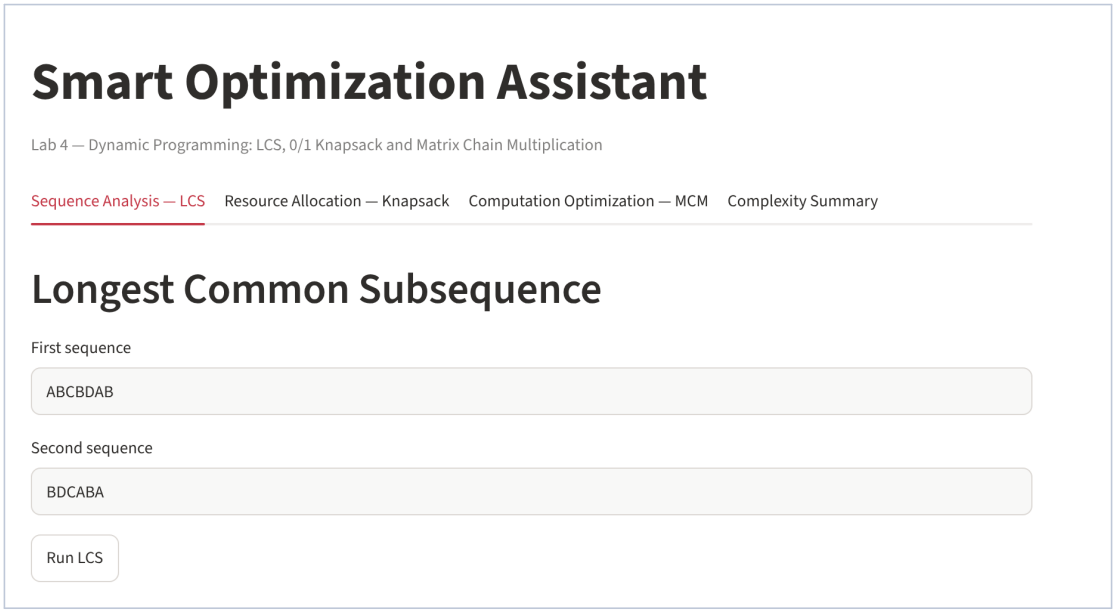
	LCS	0/1 Knapsack	Matrix Chain
State	dp[i][j] = LCS length of the first i characters of X and first j of Y	dp[i][w] = best value using the first i items with capacity w	m[i][j] = fewest multiplications for $A_i \ldots A_j$
Base case	dp[0][j] = dp[i][0] = 0	dp[0][w] = 0	m[i][i] = 0
Answer	dp[m][n]	dp[n][W]	m[1][n]

6. Application Design

The application (`app.py`) is a single Streamlit script with four tabs. Each of the three algorithm tabs follows the same pattern:

- **Input** — text boxes for the data (a string, a comma-separated list, or a number) with default sample values from the lab manual.
- **Validation** — for example, Knapsack checks that weights and values have equal length, weights are positive and values are non-negative; bad input shows an error instead of crashing.
- **Run** — a button runs the algorithm and measures time with `time.perf_counter()` and peak memory with `tracemalloc`.
- **Output** — a success message with the answer, three metric tiles (answer, execution time, peak memory), and the DP table(s).
- **Table display** — a shared helper, `show_table()`, shows each DP table as a compact scrollable data frame and as a small heatmap with the value printed in each cell, so both are easy to include in screenshots.
- **Caption** — the number of subproblems (table cells) and the time and space complexity of the run.

The fourth tab, *Complexity Summary*, shows the theoretical comparison table (Section 10).



Screenshot 1 — Application home screen: four tabs, with the LCS tab open

7. Module 1 — Sequence Analysis: Longest Common Subsequence (Question 2)

Given two sequences, LCS finds the longest sequence of characters that appears, **in order but not necessarily next to each other**, in both. For X = ABCBDAB and Y = BDCABA, BCBA is a common subsequence of length 4.

7.1 Recurrence and algorithm

```
dp[i][j] = 0                              if i = 0 or j = 0
dp[i][j] = dp[i-1][j-1] + 1               if X[i-1] == Y[j-1]
dp[i][j] = max(dp[i-1][j], dp[i][j-1])    otherwise

ALGORITHM LCS(X, Y)                 // len(X)=m, len(Y)=n
  Create table dp[0..m][0..n], initialized to 0
  FOR i = 1 TO m
    FOR j = 1 TO n
      IF X[i-1] == Y[j-1]
        dp[i][j] = dp[i-1][j-1] + 1
      ELSE
        dp[i][j] = MAX(dp[i-1][j], dp[i][j-1])
  RETURN dp[m][n], and trace back through dp[][] to build the subsequence
```

7.2 Implementation (from app.py)

```
def lcs(x, y):
    m, n = len(x), len(y)
    dp = [[0]*(n+1) for _ in range(m+1)]
    for i in range(1, m+1):
        for j in range(1, n+1):
            if x[i-1] == y[j-1]:
                dp[i][j] = dp[i-1][j-1] + 1
            else:
                dp[i][j] = max(dp[i-1][j], dp[i][j-1])
    i, j, seq = m, n, []
    while i > 0 and j > 0:
        if x[i-1] == y[j-1]:
            seq.append(x[i-1]); i -= 1; j -= 1
        elif dp[i-1][j] >= dp[i][j-1]:
            i -= 1
        else:
            j -= 1
    return dp[m][n], "".join(reversed(seq)), dp
```

The first double loop fills the table. The *traceback* loop starts at dp[m][n]: if the two characters match, that character belongs to the LCS and the walk moves diagonally; otherwise it moves to the neighbour with the larger value (up on a tie). The characters are collected backwards, so the list is reversed at the end.

7.3 Sample run: X = ABCBDAB, Y = BDCABA

X \ Y	∅	B	D	C	A	B	A
∅	0	0	0	0	0	0	0
A	0	0	0	0	1	1	1
B	0	1	1	1	1	2	2
C	0	1	1	2	2	2	2
B	0	1	1	2	2	3	3
D	0	1	2	2	2	3	3
A	0	1	2	2	3	3	4
B	0	1	2	2	3	4	4

Table 1 — Filled LCS table. The bottom-right cell is the answer.

Result: LCS length = **4**, subsequence = **BCBA**. The traceback finds four matching pairs, in order: X[2] = B with Y[1], X[3] = C with Y[3], X[4] = B with Y[5], and X[6] = A with Y[6]. Other subsequences of length 4 also exist (BDAB and BCAB); the tie-break rule in the code (move up when the values are equal) selects BCBA. The table has 7 × 6 = 42 subproblems.

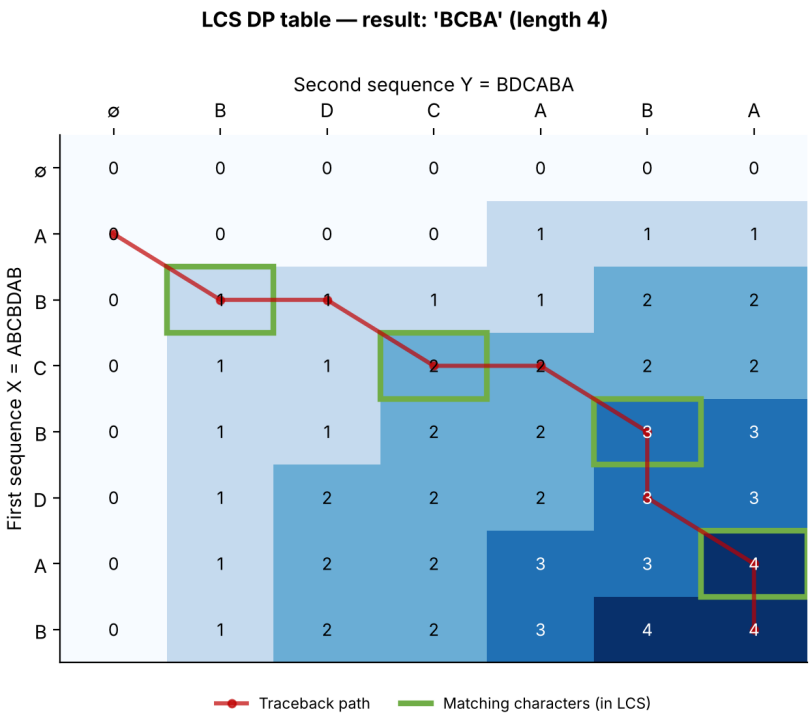


Figure 1 — LCS DP table with the traceback path (red) and matching characters (green boxes).

Smart Optimization Assistant

Lab 4 — Dynamic Programming: LCS, 0/1 Knapsack and Matrix Chain Multiplication

Sequence Analysis — LCS | Resource Allocation — Knapsack | Computation Optimization — MCM | Complexity Summary

Longest Common Subsequence

First sequence

ABCBDAB

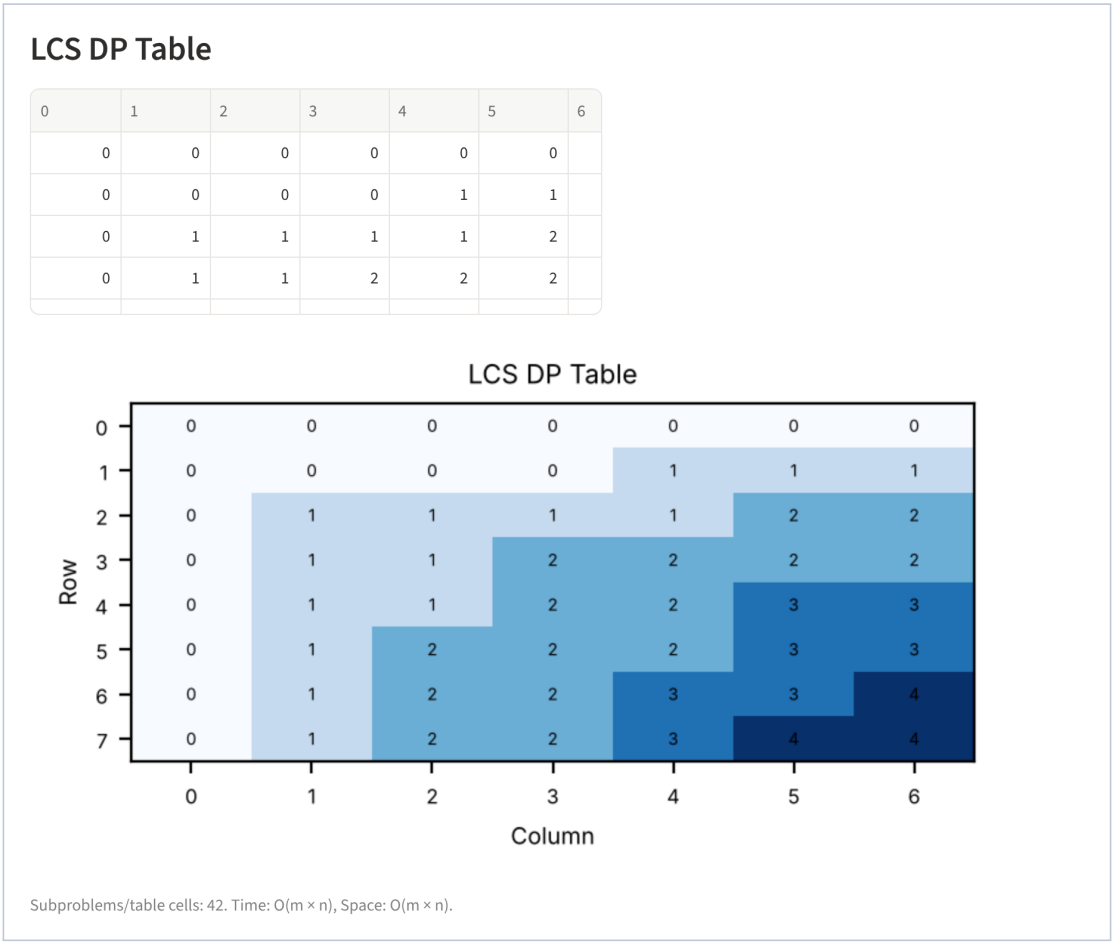
Second sequence

BDCABA

Run LCS

LCS: BCBA | Length: 4

LCS length	Execution time	Peak traced memory
4	0.000069 s	0.65 KB



Screenshot 2 — LCS tab: result BCBA, length 4, execution time, peak memory, DP table and heatmap

Complexity. Time O(m × n): each of the m × n cells is computed once in constant time. Space O(m × n) for the table; if only the length is needed, two rows are enough, giving O(min(m, n)).

8. Module 2 — Resource Allocation: 0/1 Knapsack (Question 3)

Given items with weights and values and a knapsack of fixed capacity, choose a subset — each item taken whole or not at all — that maximizes the total value without exceeding the capacity. Unlike fractional knapsack, **greedy does not work here**; DP guarantees the optimal answer.

8.1 Recurrence and algorithm

```
dp[i][w] = dp[i-1][w]                                          if weights[i-1] > w
dp[i][w] = max(values[i-1] + dp[i-1][w - weights[i-1]],        otherwise
               dp[i-1][w])                                      (take item i, or skip it)

ALGORITHM Knapsack01(weights, values, n, capacity)
  Create table dp[0..n][0..capacity], initialized to 0
  FOR i = 1 TO n
    FOR w = 0 TO capacity
      IF weights[i-1] <= w
        dp[i][w] = MAX(values[i-1] + dp[i-1][w-weights[i-1]], dp[i-1][w])
      ELSE
        dp[i][w] = dp[i-1][w]
  RETURN dp[n][capacity], and trace back to find the selected items
```

8.2 Implementation (from app.py)

```
def knapsack(weights, values, capacity):
    n = len(weights)
    dp = [[0]*(capacity+1) for _ in range(n+1)]
    for i in range(1, n+1):
        for w in range(capacity+1):
            if weights[i-1] <= w:
                dp[i][w] = max(values[i-1] + dp[i-1][w-weights[i-1]], dp[i-1][w])
            else:
                dp[i][w] = dp[i-1][w]
    chosen, w = [], capacity
    for i in range(n, 0, -1):
        if dp[i][w] != dp[i-1][w]:
            chosen.append(i-1); w -= weights[i-1]
    chosen.reverse()
    return dp[n][capacity], chosen, dp
```

In the traceback, if dp[i][w] differs from dp[i−1][w] then item i must have been taken, so it is recorded and its weight is subtracted from w; otherwise the item was skipped.

8.3 Sample run: weights 2,3,4,5 — values 3,4,5,6 — capacity 5

items \ w	0	1	2	3	4	5
0 items	0	0	0	0	0	0
item 1	0	0	3	3	3	3
item 2	0	0	3	4	4	7
item 3	0	0	3	4	5	7
item 4	0	0	3	4	5	7

Table 2 — Filled 0/1 Knapsack table. The bottom-right cell is the answer.

Result: maximum profit = **7** with **items 1 and 2** (weights 2 + 3 = 5, values 3 + 4 = 7). **Traceback:** dp[4][5] = dp[3][5] = dp[2][5] = 7, so items 4 and 3 were skipped; dp[2][5] = 7 differs from dp[1][5] = 3, so item 2 was taken and the capacity drops to 5 − 3 = 2; dp[1][2] = 3 differs from dp[0][2] = 0, so item 1 was taken. The table has 4 × 6 = 24 cells. Items 3 and 4 have higher values on their own, but neither can be combined with another item within a capacity of 5.

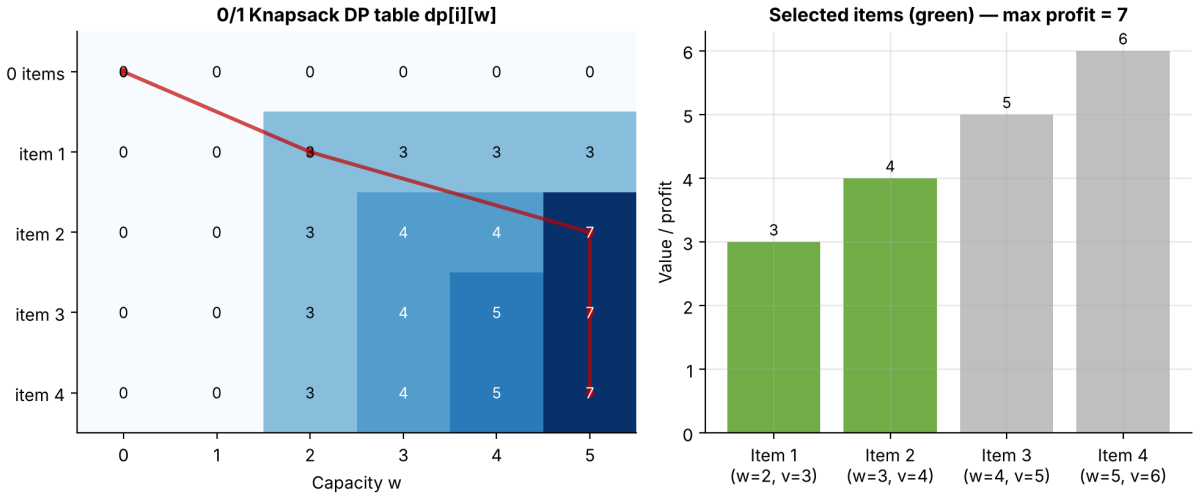


Figure 2 — 0/1 Knapsack DP table with the traceback path and the selected items (green).

Smart Optimization Assistant

Lab 4 — Dynamic Programming: LCS, 0/1 Knapsack and Matrix Chain Multiplication

Sequence Analysis — LCS | Resource Allocation — Knapsack | Computation Optimization — MCM | Complexity Summary

0/1 Knapsack

Item weights (comma-separated)

2,3,4,5

Item values/profits (comma-separated)

3,4,5,6

Knapsack capacity

5 − +

Run Knapsack

Maximum achievable profit: 7

Selected items (1-based):

```
[
  0 : 1
  1 : 2
]
```

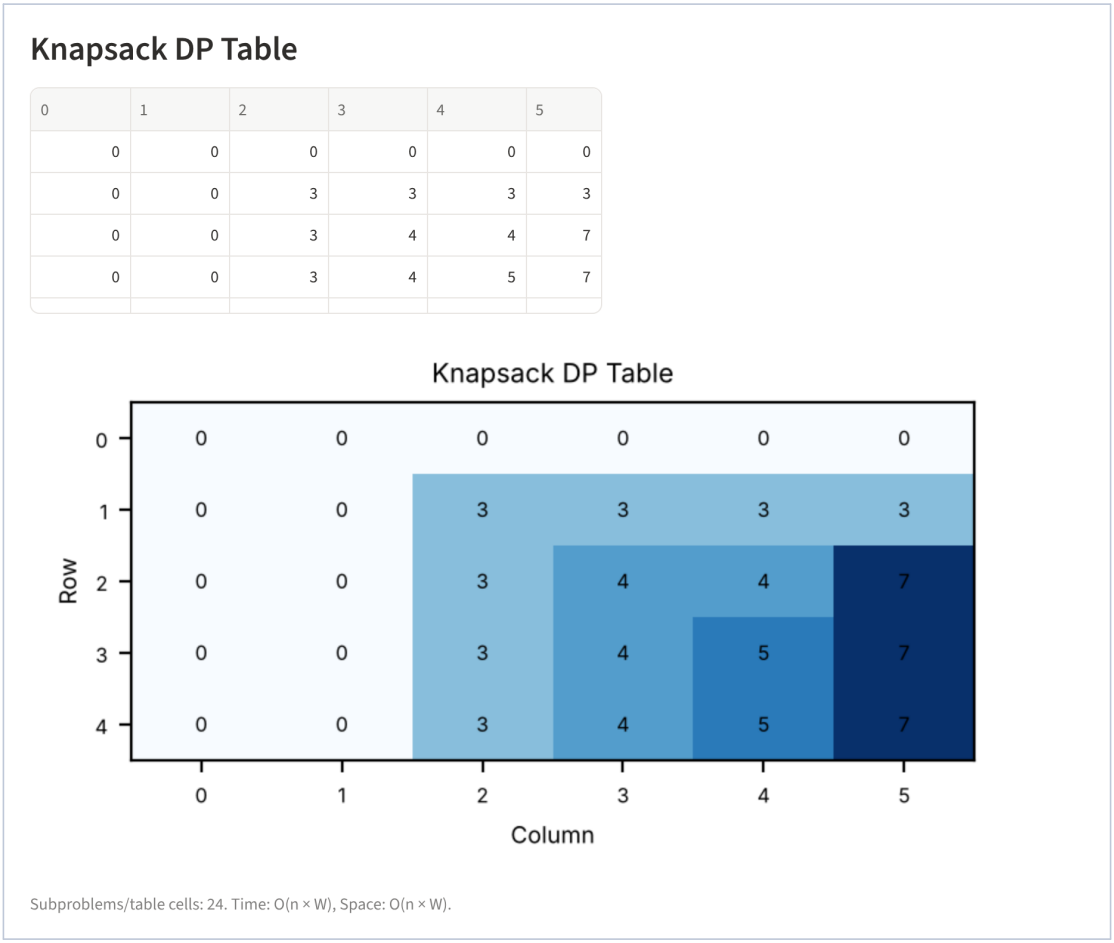
Selected weights:

```
[
  0 : 2
  1 : 3
]
```

Selected values:

```
[
  0 : 3
  1 : 4
]
```

Maximum profit	Execution time	Peak traced memory
7	0.000052 s	0.42 KB



Screenshot 3 — Knapsack tab: maximum profit 7, selected items, weights and values, DP table and heatmap

8.4 Why greedy fails for 0/1 Knapsack

In the Greedy lab, fractional knapsack takes items in decreasing value-to-weight ratio and is provably optimal because an item can be split. If the same rule is applied when items must be taken whole, it can go wrong. Using the Greedy lab's data (weights 10, 20, 30; values 60, 100, 120; capacity 50):

Method	Items taken	Total value
Fractional greedy (items may be split)	Items 1 and 2 whole + 2/3 of item 3	240
Greedy by ratio, items taken whole	Items 1 and 2 (item 3 no longer fits)	160
0/1 Knapsack DP (this lab)	Items 2 and 3	**220**

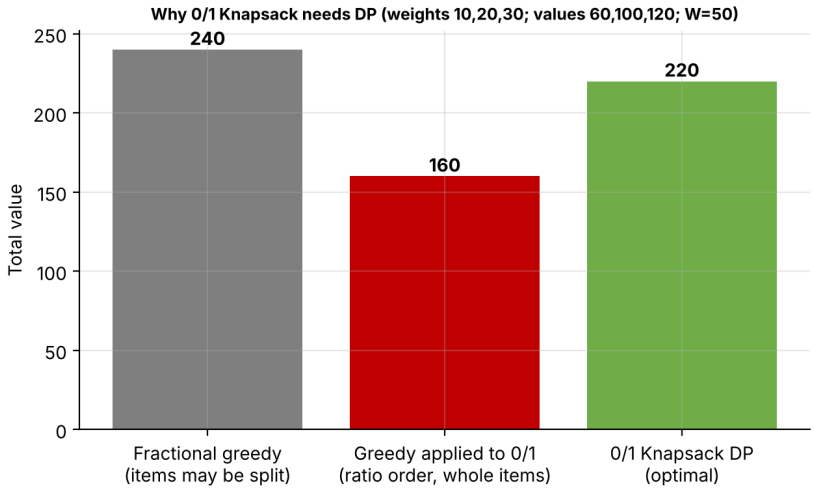


Figure 3 — Greedy by ratio gives 160 for the 0/1 problem, while DP finds the optimum of 220.

Complexity. Time O(n × W), space O(n × W) (reducible to O(W) with a 1-D array if only the value is needed). The bound is *pseudo-polynomial*: it depends on the numeric value of the capacity W, not only on the number of items.

9. Module 3 — Computation Optimization: Matrix Chain Multiplication (Question 4)

Multiplying a chain of matrices gives the same result however it is bracketed, but the **number of scalar multiplications** can differ enormously. MCM finds the bracketing with the fewest multiplications; the matrices themselves are never multiplied, only the best order is computed. The input is a dimension list p: matrix A_i has size p[i−1] × p[i].

9.1 Recurrence and algorithm

```
m[i][i] = 0
m[i][j] = min over i <= k < j of  m[i][k] + m[k+1][j] + p[i-1] * p[k] * p[j]
s[i][j] = the k that gives this minimum   (used to rebuild the bracketing)

ALGORITHM MatrixChainOrder(p)     // p[0..n] holds matrix dimensions
  n = length(p) - 1
  Create tables cost[1..n][1..n] = 0, split[1..n][1..n]
  FOR chainLength = 2 TO n
    FOR i = 1 TO n - chainLength + 1
      j = i + chainLength - 1
      cost[i][j] = INFINITY
      FOR k = i TO j-1
        q = cost[i][k] + cost[k+1][j] + p[i-1]*p[k]*p[j]
        IF q < cost[i][j]
          cost[i][j] = q
          split[i][j] = k
  RETURN cost[1][n], split table
```

9.2 Implementation (from app.py)

```
def matrix_chain(dims):
    n = len(dims)-1
    cost = [[0]*(n+1) for _ in range(n+1)]
    split = [[0]*(n+1) for _ in range(n+1)]
    for length in range(2, n+1):
        for i in range(1, n-length+2):
            j = i+length-1
            cost[i][j] = float("inf")
            for k in range(i, j):
                q = cost[i][k] + cost[k+1][j] + dims[i-1]*dims[k]*dims[j]
                if q < cost[i][j]:
                    cost[i][j], split[i][j] = q, k
    def order(i, j):
        if i == j: return f"A{i}"
        k = split[i][j]
        return "(" + order(i, k) + " × " + order(k+1, j) + ")"
    return cost[1][n], cost, split, order(1, n)
```

The chain length grows from 2 to n, so every shorter sub-chain is already solved when a longer one needs it. The nested function `order()` reads the split table recursively to produce the bracketed expression.

9.3 Sample run: dimensions 10, 30, 5, 60

This means A1 is 10 × 30, A2 is 30 × 5 and A3 is 5 × 60.

i \ j	A1	A2	A3
A1	0	1500	4500
A2		0	9000
A3			0

i \ j	A1	A2	A3
A1	0	1	2
A2		0	2
A3			0

Table 3 — Cost table m[i][j] (left) and split table s[i][j] (right); only the upper triangle is used.

Result: minimum cost = **4500** scalar multiplications; optimal order = **((A1 × A2) × A3)**. **Working:** m[1][2] = 10·30·5 = 1500 and m[2][3] = 30·5·60 = 9000. For the whole chain, splitting at k = 1 costs 0 + 9000 + 10·30·60 = 27,000, while splitting at k = 2 costs 1500 + 0 + 10·5·60 = 4500. The smaller value wins, so m[1][3] = 4500 and s[1][3] = 2. The best order is six times cheaper than the worst.

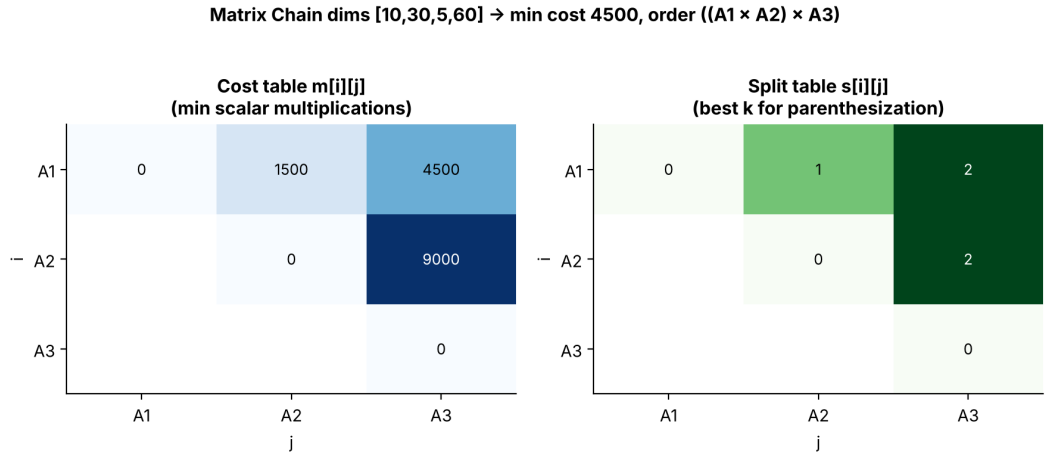
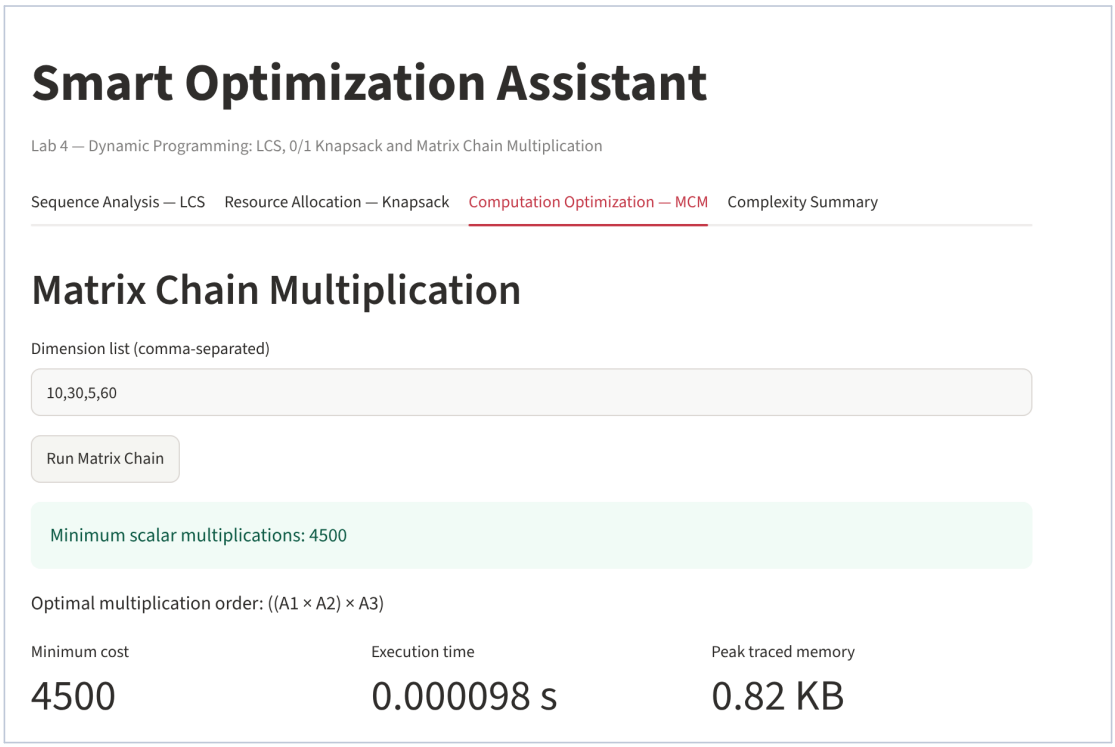
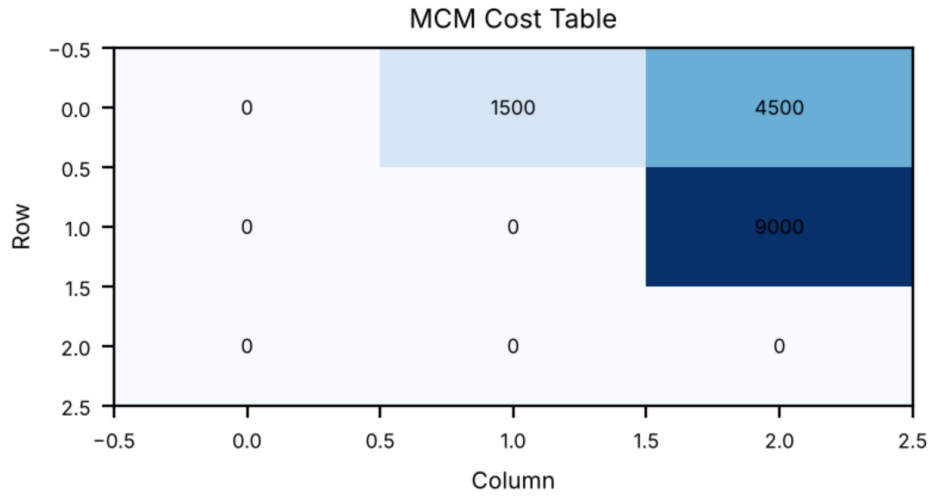


Figure 4 — Matrix Chain cost table and split table (upper triangle).



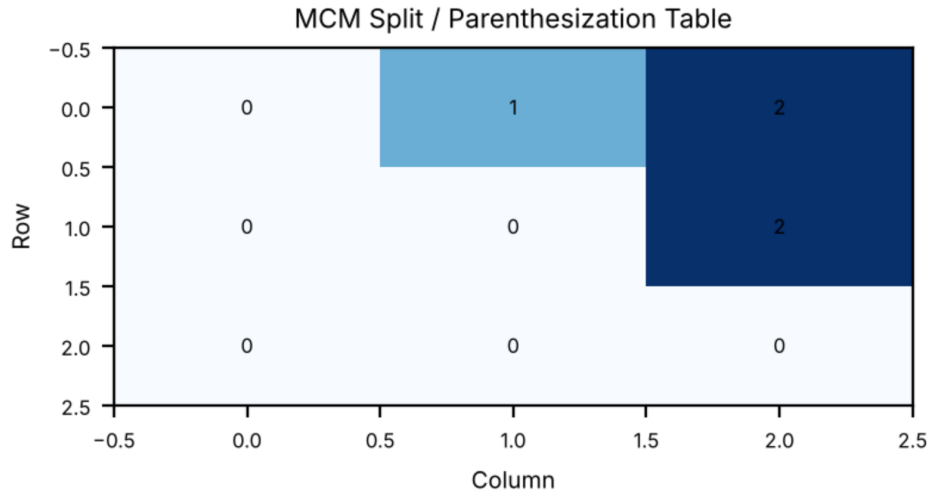
MCM Cost Table

0	1	2
0	1500	4500
0	0	9000
0	0	0



MCM Split / Parenthesization Table

0	1	2
0	1	2
0	0	2
0	0	0



Number of matrices: 3. Time: $O(n^3)$, Space: $O(n^2)$.

Screenshot 4 — Matrix Chain tab: minimum cost 4500, optimal order, cost table and split table with heatmaps

Reading the app's MCM heatmaps. The app drops the unused row 0 and column 0, so heatmap row/column **0** means matrix **A1**, 1 means A2 and 2 means A3.

Complexity. Time $O(n^3)$: there are about $n^2/2$ sub-chains and each tries up to n split points. Space $O(n^2)$ for the cost and split tables.

10. Complexity and Performance Analysis (Question 5)

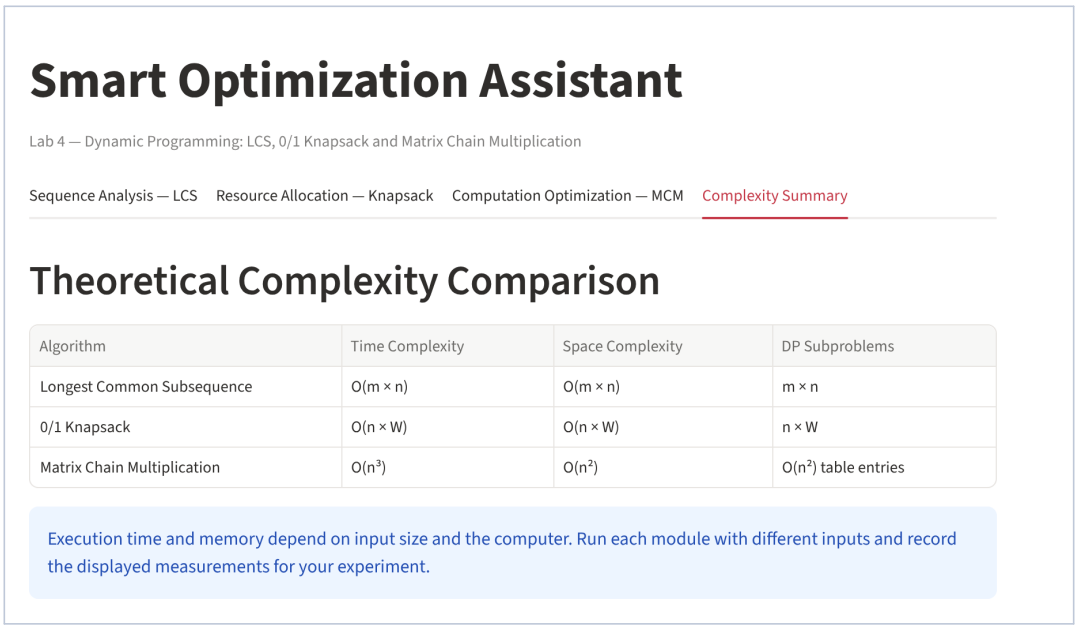
10.1 Complexity comparison table

Algorithm	Time Complexity	Space Complexity	DP Subproblems
Longest Common Subsequence	O(m × n)	O(m × n)	m × n
0/1 Knapsack	O(n × W)	O(n × W)	n × W
Matrix Chain Multiplication	$O(n^3)$	$O(n^2)$	$O(n^2)$ table entries

Table 4 — Theoretical complexity comparison (the same table appears in the app's Complexity Summary tab).

	LCS	0/1 Knapsack	Matrix Chain
Work per cell	O(1)	O(1)	O(n)
Table size	(m+1) × (n+1)	(n+1) × (W+1)	n × n (upper triangle)
Space-optimized	O(min(m, n)) for length only	O(W) with a 1-D array	not reduced in the basic version
Naive recursion	Exponential	$O(2^n)$	$\Omega(2^n)$

Table 5 — Why the bounds hold: table size × work per cell.



Screenshot 5 — Complexity Summary tab of the application

10.2 Measurements on the sample inputs

Time is the best of 200 runs measured with `time.perf_counter()`, and memory is the peak reported by `tracemalloc`, each measured in a separate run. **Environment:** Python 3.12.3 on Linux x86-64 (the machine used to prepare this report). Absolute values will differ on another computer; the growth trends will not.

Algorithm	Input	Result	Subproblems	Time	Peak memory
LCS	ABCBDAB / BDCABA	BCBA (length 4)	42	6.6 µs	0.65 KB
0/1 Knapsack	w 2,3,4,5; v 3,4,5,6; W 5	profit 7	24	2.8 µs	0.48 KB

Algorithm	Input	Result	Subproblems	Time	Peak memory
Matrix Chain	10,30,5,60	cost 4500	3 (non-trivial)	2.3 µs	1.13 KB

Table 6 — Measured results on the sample inputs.

Why the app shows larger times. Inside the application, time and memory are measured in the *same* run, with `tracemalloc` switched on. Tracing every allocation slows the code, so the times in the app's screenshots (tens to hundreds of microseconds) are higher than the times above, which were measured without tracing. The memory figures are comparable.

10.3 Measured scaling for growing input sizes

Random inputs with a fixed random seed were used. LCS: random DNA-like strings with m = n. Knapsack: random weights in [1, 50] and values in [1, 100]. MCM: random dimensions in [5, 60].

n (m = n)	Cells n × n	Time	Time per cell	Peak memory
100	10,000	1.31 ms	0.131 µs	82.97 KB
200	40,000	4.49 ms	0.112 µs	326.44 KB
300	90,000	10.73 ms	0.119 µs	725.89 KB
400	160,000	20.32 ms	0.127 µs	1.25 MB
500	250,000	32.69 ms	0.131 µs	2.10 MB
600	360,000	48.40 ms	0.134 µs	3.28 MB
700	490,000	66.17 ms	0.135 µs	4.87 MB
800	640,000	93.91 ms	0.147 µs	6.76 MB

Table 7 — LCS scaling.

Sweep	n	W	Cells n × (W+1)	Time	Peak memory
vary n	20	1000	20,020	2.77 ms	540.52 KB
vary n	40	1000	40,040	5.63 ms	1.11 MB
vary n	60	1000	60,060	8.90 ms	1.70 MB
vary n	80	1000	80,080	11.70 ms	2.09 MB
vary n	100	1000	100,100	14.45 ms	2.39 MB
vary n	120	1000	120,120	17.48 ms	2.52 MB
vary n	140	1000	140,140	20.94 ms	3.20 MB
vary n	160	1000	160,160	23.66 ms	3.54 MB
vary n	180	1000	180,180	26.52 ms	3.57 MB
vary n	200	1000	200,200	29.29 ms	4.01 MB
vary W	100	200	20,100	2.44 ms	300.52 KB
vary W	100	400	40,100	5.21 ms	761.74 KB
vary W	100	600	60,100	8.19 ms	1.27 MB
vary W	100	800	80,100	10.86 ms	1.87 MB
vary W	100	1000	100,100	14.42 ms	2.50 MB
vary W	100	1200	120,100	17.81 ms	3.15 MB
vary W	100	1400	140,100	21.30 ms	3.81 MB
vary W	100	1600	160,100	24.35 ms	4.49 MB
vary W	100	1800	180,100	28.14 ms	5.19 MB
vary W	100	2000	200,100	32.21 ms	5.90 MB

Table 8 — 0/1 Knapsack scaling (n varied with W = 1000; W varied with n = 100).

n (matrices)	Subproblems n(n−1)/2	Time	Peak memory
10	45	25.3 µs	5.00 KB
20	190	145.1 µs	15.21 KB
30	435	461.6 µs	30.65 KB
40	780	1.04 ms	55.13 KB
50	1,225	1.96 ms	83.76 KB
60	1,770	3.34 ms	119.77 KB
70	2,415	5.37 ms	161.84 KB
80	3,160	7.82 ms	211.40 KB
90	4,005	11.04 ms	266.09 KB
100	4,950	15.24 ms	327.09 KB
110	5,995	20.23 ms	395.29 KB
120	7,140	26.60 ms	465.23 KB

Table 9 — Matrix Chain Multiplication scaling.

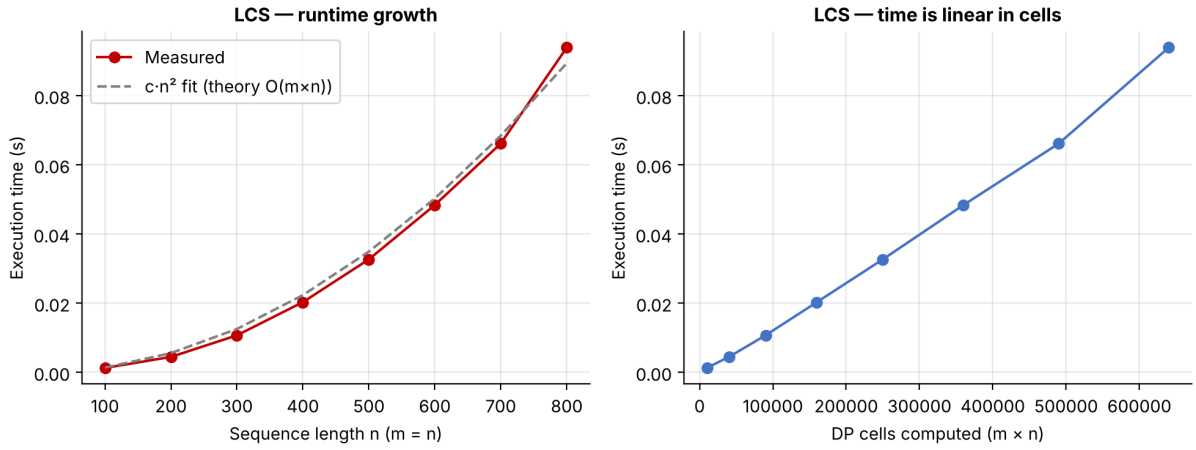


Figure 5 — LCS — runtime against n (left, with fitted c·n² curve) and against the number of cells (right).

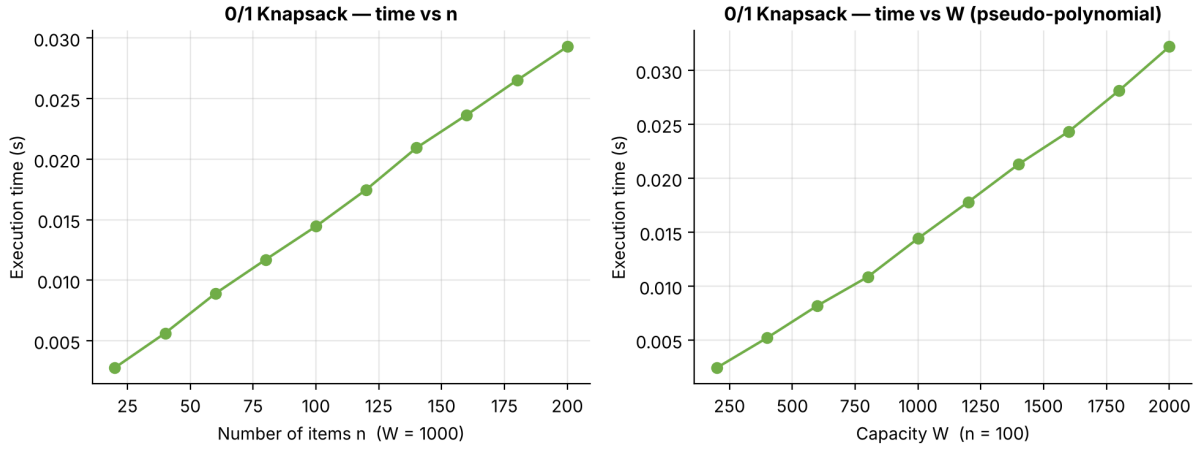


Figure 6 — 0/1 Knapsack — runtime against n with W fixed (left) and against W with n fixed (right).

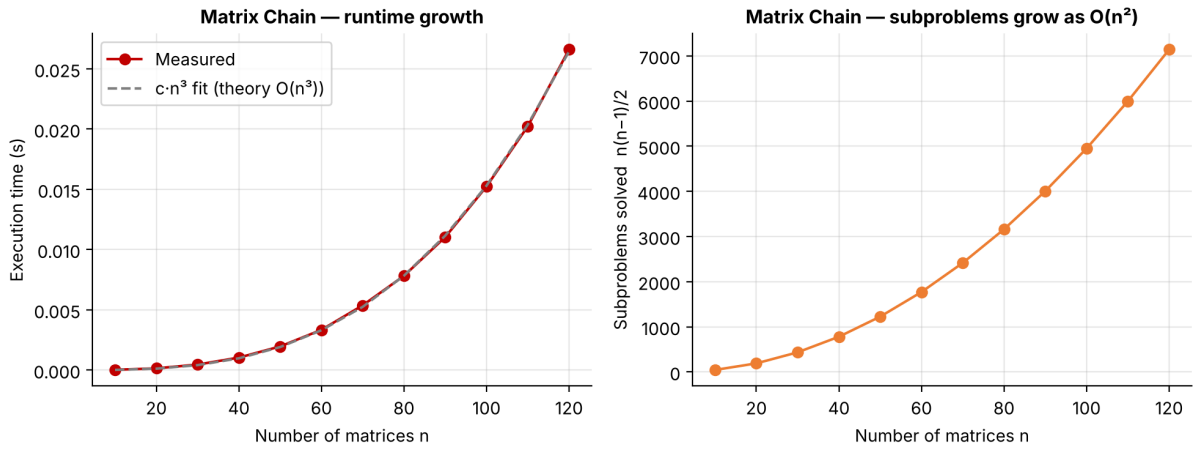


Figure 7 — Matrix Chain — runtime (left, with fitted c·n³ curve) and subproblem count (right).

10.4 Measured growth against theory

Experiment	Theoretical exponent	Measured log-log slope
LCS, m = n (time vs n)	2	2.07
0/1 Knapsack, W fixed (time vs n)	1	1.03
0/1 Knapsack, n fixed (time vs W)	1	1.12
Matrix Chain (time vs n)	3	2.83

Table 10 — Fitted growth exponents.

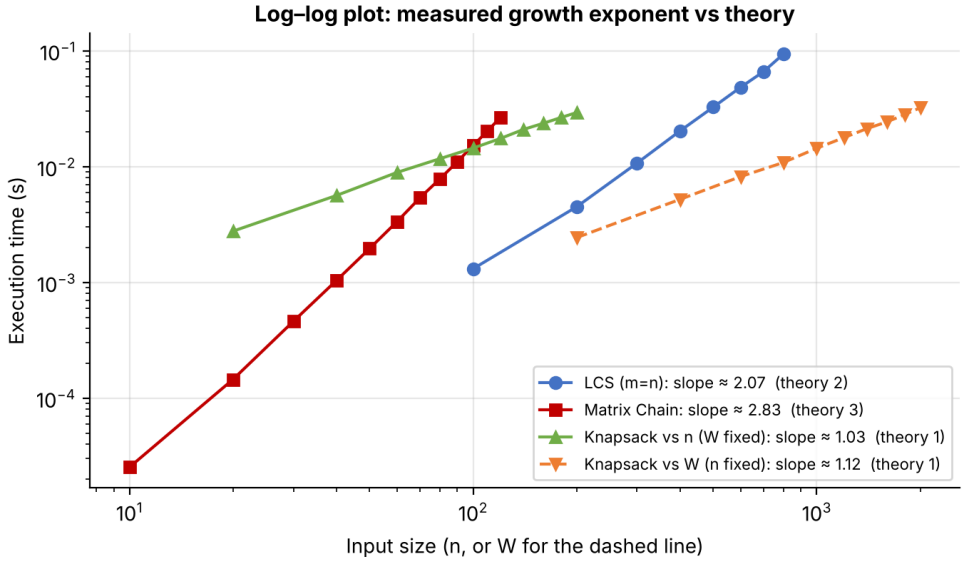


Figure 8 — Log–log plot: measured slopes against the theoretical exponents 1, 2 and 3.

The measured slopes (2.07, 1.03, 1.12 and 2.83) agree closely with the theoretical O(m × n), O(n × W) and O(n³). The MCM slope is a little below 3 because lower-order costs (loop set-up and allocating the tables) are still noticeable at these sizes.

10.5 Space usage

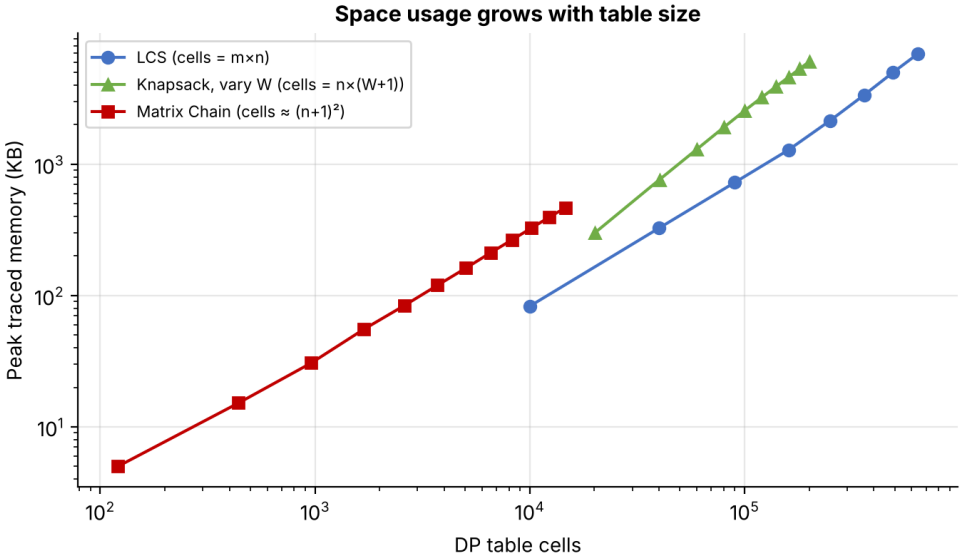


Figure 9 — Peak traced memory against the number of DP table cells (log–log).

Memory grows with the table size. At n = 800 the LCS table needs about 6.76 MB; the Knapsack table at W = 2000 needs about 5.90 MB; the two MCM tables at n = 120 need only 465.23 KB because they hold just $O(n^2)$ entries. The figures are larger than the raw table size in C because Python stores every list slot as a pointer and every integer above 256 as a separate object.

11. Comparison with the Naive Recursive Approach

The naive recursive LCS follows the recurrence directly, with no table:

```
def lcs_rec(x, y, i, j):
    if i == 0 or j == 0: return 0
    if x[i-1] == y[j-1]: return 1 + lcs_rec(x, y, i-1, j-1)
    return max(lcs_rec(x, y, i-1, j), lcs_rec(x, y, i, j-1))
```

Because the subproblem (i, j) is reached along many different paths, it is recomputed over and over: this is the *overlapping subproblems* property. In the worst case, when the two strings share no characters, every call makes two more calls. For two strings of length n the number of calls is exactly $2 \cdot C(2n, n) - 1$, which grows by almost 4 times for each extra character (the measured call counts below match this formula for every n from 2 to 13). The DP table, in contrast, fills just n × n cells.

n	Recursive calls	Recursive time	DP cells (n × n)	DP time	DP faster by
2	11	15.0 µs	4	1.5 µs	10×
3	39	5.6 µs	9	2.3 µs	2×
4	139	15.6 µs	16	3.4 µs	5×
5	503	75.7 µs	25	4.5 µs	17×
6	1,847	195.3 µs	36	6.0 µs	33×
7	6,863	796.9 µs	49	7.9 µs	101×
8	25,739	2.77 ms	64	9.6 µs	289×
9	97,239	10.20 ms	81	11.9 µs	858×
10	369,511	38.20 ms	100	14.1 µs	2,709×
11	1,410,863	153.74 ms	121	16.4 µs	9,362×
12	5,408,311	709.12 ms	144	19.4 µs	36,609×
13	20,801,199	2.316 s	169	22.4 µs	103,219×

Table 11 — Naive recursion against DP for LCS, worst case (no common characters; strings of 'a' and 'b').

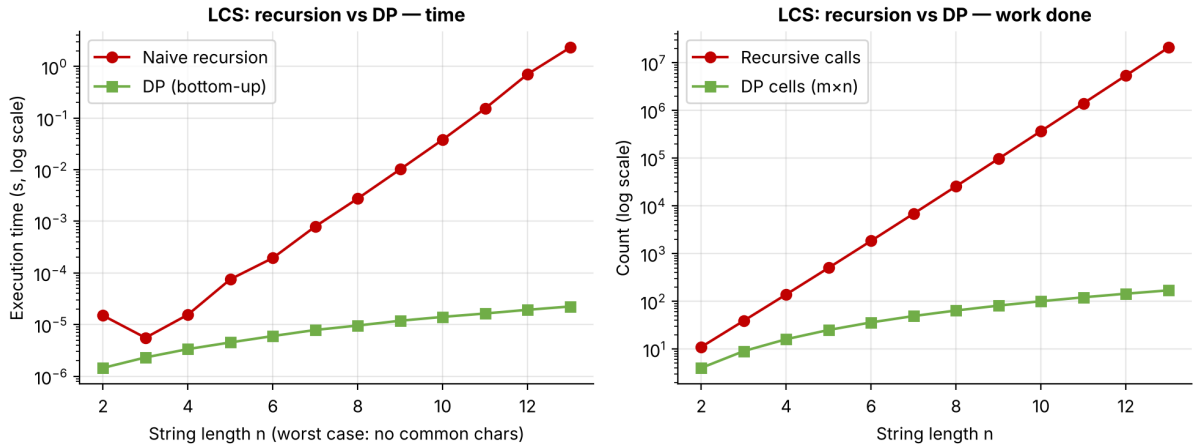


Figure 10 — LCS — naive recursion against the DP table, on a log scale (time on the left, work done on the right).

At n = 13 the recursive version needs 20,801,199 calls and 2.316 s, while the DP version computes 169 cells in 22.4 µs — about 103,219 times faster. The same trade-off applies to the other two problems: naive recursion for 0/1 Knapsack tries include/exclude for every item ($O(2^n)$ calls), and naive recursion for MCM is at least exponential in n. A top-down recursion with memoization also reaches polynomial time by caching results; this lab uses the bottom-up table because it makes the DP table directly visible.

12. Visualizations (Question 6)

The lab asks for visualizations of the DP tables, optimization results, performance statistics and experimental observations for different problem sizes. They are provided at two levels:

- **Inside the application** — each DP table is shown as a data frame and as a heatmap (Screenshots 2, 3 and 4).
- **Generated with Matplotlib for this report** — the figures listed below, built from the same algorithm code as the app.

Figure	Visualization	Category	Section
1	LCS DP table heatmap with traceback path	DP table / optimization result	7
2	0/1 Knapsack DP table with traceback and selected items	DP table / optimization result	8
3	Greedy vs DP bar chart for 0/1 Knapsack	Optimization result	8
4	Matrix Chain cost and split tables	DP table / optimization result	9
5, 6, 7	Runtime growth for LCS, Knapsack, Matrix Chain	Performance, different sizes	10
8	Log–log growth exponents against theory	Performance, observations	10
9	Peak memory against table size	Performance statistics	10
10	Naive recursion against DP (time and calls)	Experimental observation	11

A separate document, *Lab4_2_Visualizations.pdf*, presents all of these with an explanation under each chart, together with the application screenshots.

13. Practical Applications

Algorithm	Where it is used	How the DP helps
LCS	File comparison and version-control diff tools; bioinformatics (comparing DNA or protein sequences); plagiarism and similarity checking; merging edits to a document	The longest common part shows what stayed the same, so everything outside it is reported as added or removed
0/1 Knapsack	Budgeting and choosing projects under a fixed budget; loading cargo within a weight limit; selecting items under a time or memory limit	Finds the combination of whole items with the highest total value that fits the limit
Matrix Chain Multiplication	Compilers and expression evaluators; graphics pipelines that combine many transformation matrices; scientific and machine-learning code that multiplies long matrix chains (for example NumPy's `numpy.linalg.multi_dot` chooses the multiplication order)	Chooses the bracketing with the fewest scalar multiplications, which can be many times cheaper than a left-to-right order

14. Key Observations

1. Measured growth matches the theory. The fitted log–log slopes are 2.07 for LCS (theory 2), 1.03 and 1.12 for Knapsack (theory 1) and 2.83 for Matrix Chain (theory 3).

2. Time is proportional to the number of cells. For LCS and Knapsack the time per table cell stays between about 0.11 and 0.16 µs across all sizes tested, so doubling the work (cells) doubles the time. The small upward drift for large LCS tables is expected, because bigger tables are less cache-friendly.

3. Doubling the input does not simply double the cost. Doubling both LCS strings (n = 400 to 800) quadrupled the cells and raised the time by 4.6× (theory 4×). Memory rose by 5.4×, a little more than 4×, which is probably because larger DP values no longer fit in CPython's cache of small integers and each becomes a separate object.

4. Knapsack is pseudo-polynomial. Doubling the capacity from W = 1000 to 2000 raised the time by 2.2× and the memory by 2.4×, even though W needs only one more bit to write down. A very large capacity therefore makes the table very large.

5. MCM is the most expensive of the three for large n. Its time grows cubically; it takes 26.60 ms at n = 120. Its memory, however, stays small (465.23 KB) because only $O(n^2)$ entries are stored.

6. The order of multiplication matters. For dimensions 10, 30, 5, 60 the best order costs 4500 multiplications and the worst costs 27,000 — six times more.

7. DP removes exponential work. At n = 13 naive recursion is about 103,219 times slower than the DP table and makes 123,084 times as many steps as there are cells.

8. Greedy is not enough for 0/1 Knapsack. Greedy by ratio gives 160 on the lab data where DP finds 220.

9. Timing at very small scale is noisy. Differences of a few microseconds or a few percent are within run-to-run variation; this is why the experiments use the best of several runs and why growth *trends* are more reliable than any single reading.

15. Limitations and Future Work

- **Measurement overhead.** The app measures time and memory in the same run with `tracemalloc` on, which inflates the displayed time (see Section 10.2). Measuring time and memory in two separate runs, as done for the experiments here, would give cleaner numbers.
- **Heatmap readability.** The heatmaps use a fixed small size and small fonts, so very large tables become hard to read; the scrollable data frame is better for large inputs.

- **Cosmetic axis labels.** In the MCM heatmaps the axes show 0.5-step ticks and numbers 0–2 instead of A1–A3. Setting integer tick labels would fix this.
- **No input size limit.** A very long string or a very large knapsack capacity can use a lot of memory. A sensible maximum could be added, together with a space-optimized mode (two rows for LCS, a 1-D array for Knapsack) when only the value is needed.
- **Integer inputs only.** Knapsack weights and capacity must be whole numbers, because they index the table.
- **Possible extensions.** A runtime-growth tab inside the app, a memoized recursion mode, exporting tables to CSV, and deploying the app on Streamlit Community Cloud.

16. Conclusion

The Smart Optimization Assistant implements LCS, 0/1 Knapsack and Matrix Chain Multiplication with bottom-up Dynamic Programming and presents each result with its DP table, heatmap, execution time and memory use. The sample runs reproduce the expected answers (LCS of length 4, a Knapsack profit of 7, and a Matrix Chain cost of 4500). The experiments confirm the theoretical complexity — quadratic growth for LCS, linear growth in n and in W for Knapsack, and cubic growth for Matrix Chain — and show that tabulation turns exponential recursive solutions into polynomial ones, in the LCS test by a factor of about 100,000 at only n = 13. The Knapsack comparison with the Greedy lab shows why Dynamic Programming is needed when a greedy choice is not safe. Together, the application, the measurements and the visualizations meet the objectives of Lab 4.

Appendix A — Complete Source Code

A.1 requirements.txt

```
streamlit
numpy
matplotlib
pandas
```

A.2 app.py

```
import streamlit as st
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import time
import tracemalloc

st.set_page_config(page_title="Smart Optimization Assistant", page_icon="□", layout="wide")
st.title("Smart Optimization Assistant")
st.caption("Lab 4 — Dynamic Programming: LCS, 0/1 Knapsack and Matrix Chain Multiplication")

def lcs(x, y):
    m, n = len(x), len(y)
    dp = [[0]*(n+1) for _ in range(m+1)]
    for i in range(1, m+1):
        for j in range(1, n+1):
            if x[i-1] == y[j-1]:
                dp[i][j] = dp[i-1][j-1] + 1
            else:
                dp[i][j] = max(dp[i-1][j], dp[i][j-1])
    i, j, seq = m, n, []
    while i > 0 and j > 0:
        if x[i-1] == y[j-1]:
            seq.append(x[i-1]); i -= 1; j -= 1
        elif dp[i-1][j] >= dp[i][j-1]:
            i -= 1
        else:
            j -= 1
    return dp[m][n], "".join(reversed(seq)), dp

def knapsack(weights, values, capacity):
    n = len(weights)
    dp = [[0]*(capacity+1) for _ in range(n+1)]
    for i in range(1, n+1):
        for w in range(capacity+1):
            if weights[i-1] <= w:
                dp[i][w] = max(values[i-1] + dp[i-1][w-weights[i-1]], dp[i-1][w])
            else:
                dp[i][w] = dp[i-1][w]
    chosen, w = [], capacity
    for i in range(n, 0, -1):
        if dp[i][w] != dp[i-1][w]:
            chosen.append(i-1); w -= weights[i-1]
    chosen.reverse()
    return dp[n][capacity], chosen, dp

def matrix_chain(dims):
    n = len(dims)-1
    cost = [[0]*(n+1) for _ in range(n+1)]
    split = [[0]*(n+1) for _ in range(n+1)]
    for length in range(2, n+1):
        for i in range(1, n-length+2):
            j = i+length-1
            cost[i][j] = float("inf")
            for k in range(i, j):
                q = cost[i][k] + cost[k+1][j] + dims[i-1]*dims[k]*dims[j]
                if q < cost[i][j]:
                    cost[i][j], split[i][j] = q, k
    def order(i, j):
        if i == j: return f"A{i}"
        k = split[i][j]
        return "(" + order(i, k) + " × " + order(k+1, j) + ")"
    return cost[1][n], cost, split, order(1, n)

def show_table(table, title):
    st.subheader(title)
    df = pd.DataFrame(table)

    # Compact, fixed-size table. Scroll inside it if it has many rows/columns.
    st.dataframe(
        df,
        width=480,
        height=190,
        hide_index=True,
        column_config={
            col: st.column_config.NumberColumn(width="small")
            for col in df.columns
        },
    )

    # Small heatmap for screenshots.
    arr = np.array(table, dtype=float)
    fig, ax = plt.subplots(figsize=(4, 2.2), dpi=80)
    im = ax.imshow(arr, cmap="Blues", aspect="auto", interpolation="nearest")
```

```
    label_size = 5 if arr.size > 50 else 6

    for r in range(arr.shape[0]):
        for c in range(arr.shape[1]):
            ax.text(c, r, str(table[r][c]),
                    ha="center", va="center", fontsize=label_size)

    ax.set_title(title, fontsize=8)
    ax.tick_params(axis="both", labelsize=6)
    ax.set_xlabel("Column", fontsize=7)
    ax.set_ylabel("Row", fontsize=7)
    fig.tight_layout(pad=0.5)
    st.pyplot(fig, use_container_width=False)
    plt.close(fig)

tab1, tab2, tab3, tab4 = st.tabs(["Sequence Analysis — LCS", "Resource Allocation — Knapsack", "Computation
      Optimization — MCM", "Complexity Summary"])

with tab1:
    st.header("Longest Common Subsequence")
    x = st.text_input("First sequence", "ABCBDAB")
    y = st.text_input("Second sequence", "BDCABA")
    if st.button("Run LCS"):
        start = time.perf_counter(); tracemalloc.start()
        length, seq, table = lcs(x, y)
        current, peak = tracemalloc.get_traced_memory(); tracemalloc.stop()
        elapsed = time.perf_counter()-start
        st.success(f"LCS: {seq} | Length: {length}")
        c1,c2,c3 = st.columns(3)
        c1.metric("LCS length", length); c2.metric("Execution time", f"{elapsed:.6f} s"); c3.metric("Peak traced
              memory", f"{peak/1024:.2f} KB")
        show_table(table, "LCS DP Table")
        st.caption(f"Subproblems/table cells: {len(x)*len(y)}. Time: O(m × n), Space: O(m × n).")

with tab2:
    st.header("0/1 Knapsack")
    weights_text = st.text_input("Item weights (comma-separated)", "2,3,4,5")
    values_text = st.text_input("Item values/profits (comma-separated)", "3,4,5,6")
    capacity = st.number_input("Knapsack capacity", min_value=0, value=5, step=1)
    if st.button("Run Knapsack"):
        try:
            weights = [int(v.strip()) for v in weights_text.split(",") if v.strip()]
            values = [int(v.strip()) for v in values_text.split(",") if v.strip()]
            if not weights or len(weights) != len(values) or any(w <= 0 for w in weights) or any(v < 0 for v in
                  values):
                st.error("Enter equal numbers of positive weights and non-negative values.")
            else:
                start=time.perf_counter(); tracemalloc.start()
                profit, chosen, table=knapsack(weights, values, int(capacity))
                current, peak=tracemalloc.get_traced_memory(); tracemalloc.stop()
                elapsed=time.perf_counter()-start
                st.success(f"Maximum achievable profit: {profit}")
                st.write("Selected items (1-based):", [i+1 for i in chosen])
                st.write("Selected weights:", [weights[i] for i in chosen])
                st.write("Selected values:", [values[i] for i in chosen])
                c1,c2,c3=st.columns(3)
                c1.metric("Maximum profit", profit); c2.metric("Execution time", f"{elapsed:.6f} s"); c3.metric("Peak
                      traced memory", f"{peak/1024:.2f} KB")
                show_table(table, "Knapsack DP Table")
                st.caption(f"Subproblems/table cells: {len(weights)* (int(capacity)+1)}. Time: O(n × W), Space: O(n ×
                      W).")
        except ValueError:
            st.error("Please enter weights and values as comma-separated integers.")

with tab3:
    st.header("Matrix Chain Multiplication")
    dims_text=st.text_input("Dimension list (comma-separated)", "10,30,5,60")
    if st.button("Run Matrix Chain"):
        try:
            dims=[int(v.strip()) for v in dims_text.split(",") if v.strip()]
            if len(dims)<2 or any(d<=0 for d in dims):
                st.error("Enter at least two positive dimensions.")
            else:
                start=time.perf_counter(); tracemalloc.start()
                cost, table, split, parens=matrix_chain(dims)
                current, peak=tracemalloc.get_traced_memory(); tracemalloc.stop()
                elapsed=time.perf_counter()-start
                st.success(f"Minimum scalar multiplications: {cost}")
                st.write("Optimal multiplication order:", parens)
                c1,c2,c3=st.columns(3)
                c1.metric("Minimum cost", cost); c2.metric("Execution time", f"{elapsed:.6f} s"); c3.metric("Peak
                      traced memory", f"{peak/1024:.2f} KB")
                n=len(dims)-1
                show_table([row[1:] for row in table[1:]], "MCM Cost Table")
                show_table([row[1:] for row in split[1:]], "MCM Split / Parenthesization Table")
                st.caption(f"Number of matrices: {n}. Time: O(n³), Space: O(n²).")
```

```
        except ValueError:
            st.error("Please enter dimensions as comma-separated positive integers.")

with tab4:
    st.header("Theoretical Complexity Comparison")
    data=[
        ["Longest Common Subsequence", "O(m × n)", "O(m × n)", "m × n"],
        ["0/1 Knapsack", "O(n × W)", "O(n × W)", "n × W"],
        ["Matrix Chain Multiplication", "O(n³)", "O(n²)", "O(n²) table entries"],
    ]
    st.dataframe(pd.DataFrame(data, columns=["Algorithm","Time Complexity","Space Complexity","DP Subproblems"]),
          hide_index=True, use_container_width=True)
    st.info("Execution time and memory depend on input size and the computer. Run each module with different inputs
          and record the displayed measurements for your experiment.")
```

Appendix B — Experiment Script

This script produced the measurements in Sections 10 and 11. It imports the three algorithm functions (copied unchanged from `app.py` into `algos.py`) and writes the results to `results.json`. Running it on another computer will give different absolute times but the same growth trends.

```
import random, time, tracemalloc, json, sys
from algos import lcs, knapsack, matrix_chain
sys.setrecursionlimit(100000)
random.seed(42)

def best_time(fn, reps=3):
    best=1e9
    for _ in range(reps):
        t=time.perf_counter(); fn(); best=min(best,time.perf_counter()-t)
    return best
def peak_mem(fn):
    tracemalloc.start(); fn(); _,p=tracemalloc.get_traced_memory(); tracemalloc.stop(); return p/1024

R={}
# ---- Sample inputs (same as app defaults)
x,y="ABCBDAB","BDCABA"
L,seq,t=lcs(x,y)
P,ch,kt=knapsack([2,3,4,5],[3,4,5,6],5)
C,ct,sp,par=matrix_chain([10,30,5,60])
R["sample"]=dict(

        lcs=dict(len=L,seq=seq,cells=len(x)*len(y),time=best_time(lambda:lcs(x,y),200),mem=peak_mem(lambda:lcs(x,y)),t
        able=t),
  knap=dict(profit=P,chosen=[i+1 for i in ch],cells=4*6,time=best_time(lambda:knapsack([2,3,4,5],[3,4,5,6],5),200),mem
        =peak_mem(lambda:knapsack([2,3,4,5],[3,4,5,6],5)),table=kt),
  mcm=dict(cost=C,order=par,cells=3*2//2,time=best_time(lambda:matrix_chain([10,30,5,60]),200),mem=peak_mem(lambda:mat
        rix_chain([10,30,5,60])),cost_t=[r[1:] for r in ct[1:]],split_t=[r[1:] for r in sp[1:]]))

# ---- LCS scaling (m=n)
ns=[100,200,300,400,500,600,700,800]
R["lcs"]=[]
for n in ns:
    a="".join(random.choice("ACGT") for _ in range(n)); b="".join(random.choice("ACGT") for _ in range(n))
    R["lcs"].append(dict(n=n,cells=n*n,time=best_time(lambda:lcs(a,b)),mem=peak_mem(lambda:lcs(a,b))))
# ---- Knapsack: vary n (W=1000)
R["knap_n"]=[]
for n in [20,40,60,80,100,120,140,160,180,200]:
    w=[random.randint(1,50) for _ in range(n)]; v=[random.randint(1,100) for _ in range(n)]
    R["knap_n"].append(dict(n=n,W=1000,cells=n*1001,time=best_time(lambda:knapsack(w,v,1000)),mem=peak_mem(lambda:knap
          sack(w,v,1000))))
# ---- Knapsack: vary W (n=100)
R["knap_W"]=[]
w=[random.randint(1,50) for _ in range(100)]; v=[random.randint(1,100) for _ in range(100)]
for W in [200,400,600,800,1000,1200,1400,1600,1800,2000]:
    R["knap_W"].append(dict(n=100,W=W,cells=100*(W+1),time=best_time(lambda:knapsack(w,v,W)),mem=peak_mem(lambda:knaps
          ack(w,v,W))))
# ---- MCM scaling
R["mcm"]=[]
for n in [10,20,30,40,50,60,70,80,90,100,110,120]:
    d=[random.randint(5,60) for _ in range(n+1)]
    R["mcm"].append(dict(n=n,subproblems=n*(n-1)//2,time=best_time(lambda:matrix_chain(d),9),mem=peak_mem(lambda:matri
          x_chain(d))))

# ---- Naive recursive LCS vs DP (worst case: no matching chars)
calls=[0]
def lcs_rec(x,y,i,j):
    calls[0]+=1
    if i==0 or j==0: return 0
    if x[i-1]==y[j-1]: return 1+lcs_rec(x,y,i-1,j-1)
    return max(lcs_rec(x,y,i-1,j),lcs_rec(x,y,i,j-1))
R["rec"]=[]
for n in range(2,14):
    a="a"*n; b="b"*n
    calls[0]=0; t=time.perf_counter(); lcs_rec(a,b,n,n); tr=time.perf_counter()-t; c=calls[0]
    td=best_time(lambda:lcs(a,b),20)
    R["rec"].append(dict(n=n,calls=c,rec_time=tr,dp_cells=n*n,dp_time=td))

# ---- Greedy-vs-DP knapsack counterexample (Lab 3 data)
items=[(10,60),(20,100),(30,120)]; cap=50
srt=sorted(items,key=lambda it:it[1]/it[0],reverse=True); rem=cap; gv=0; gfrac=[]
for wt,vl in srt:
    if rem<=0: break
    if wt<=rem: gv+=vl; rem-=wt; gfrac.append(1.0)
    else: gv+=vl*rem/wt; gfrac.append(rem/wt); rem=0
g01=0; rem=cap
for wt,vl in srt:
    if wt<=rem: g01+=vl; rem-=wt
dpv,dch,_=knapsack([10,20,30],[60,100,120],50)
```

```
R["greedy"]=dict(fractional=gv,greedy_01=g01,dp_01=dpv,dp_items=[i+1 for i in dch])
json.dump(R,open("results.json","w"),indent=1,default=str)
print(json.dumps({k:R[k] for k in ["greedy"]}))
print("sample",{k:{a:b for a,b in v.items() if a not in('table','cost_t','split_t')} for k,v in R["sample"].items()})
for k in ["lcs","knap_n","knap_W","mcm","rec"]:
    print(k); [print("  ",r) for r in R[k]]
```